

From Classical Single-Marker QTL Mapping to Multi-Marker Shrinkage: Analysis and Simulation Using Cotton MAGIC Genotypes

Shiyan Miao

2026-03-10

Table of contents

1	Introduction	1
2	Theoretical Framework	2
2.1	Classical QTL mapping logic	2
2.2	Why the present dataset requires a modern extension	2
3	Data and Setup	2
3.1	Required packages	2
3.2	File paths	2
3.3	Import data	3
3.4	Inspect dimensions	3
3.5	Add marker names to the genotype matrix	3
3.6		4
3.7	Merge phenotype and genotype	5
4	Descriptive Analysis	5
4.1	Trait names	5
4.2	Summary statistics	5
4.3	Season-specific summaries	5
4.4	Trait distributions	6
4.5	Trait boxplots by season	8
4.6	Correlation among traits	10

1 Introduction

This report develops an analysis and simulation workflow for cotton genotype and phenotype data downloaded from the CSIRO collection. The workflow is designed to connect the classical QTL mapping literature with a modern high-dimensional biostatistical analysis strategy. The theoretical starting point is the classical regression-based QTL mapping tradition associated with single-marker and interval-based approaches, especially Haley and Knott’s regression approximation and Broman’s broader framework of QTL detection, interval mapping, and multiple-QTL model selection. The empirical motivation is that the present dataset is not a simple textbook backcross or F2 dataset, but a more realistic high-dimensional marker dataset with multiple phenotypic measurements across seasons. Therefore, this report treats single-marker regression as a baseline and uses penalised multi-marker methods as the main analysis framework.

The goals of the report are fourfold. First, it describes the structure of the genotype, map, and phenotype data. Second, it performs basic quality checks and descriptive summaries. Third, it conducts a baseline single-

marker scan for each trait while controlling for seasonal variation. Fourth, it proposes an anchored simulation framework that preserves the real genotype architecture and uses simulated phenotypes to compare the performance of alternative QTL detection methods under varying numbers of causal loci, signal strengths, and seasonal effects.

2 Theoretical Framework

2.1 Classical QTL mapping logic

The literature review established a progression from traditional phenotypic selection to marker-assisted selection, and from there to QTL detection as the statistical foundation of marker-assisted selection. In the classical framework, a marker is useful not because it is necessarily causal, but because it carries information about an unobserved functional locus or QTL through linkage. Under simple crossing designs, such as backcross or F2 populations, marker genotype differences can be translated into phenotypic mean differences, and those differences can be analysed through regression.

Haley and Knott showed that QTL mapping can be approximated by a regression framework rather than relying only on full maximum-likelihood routines. Broman later situated this logic within a broader progression from marker-wise analysis to interval mapping, and then to multiple-QTL and model-selection frameworks. The key implication for the present project is that single-marker regression remains theoretically useful as a baseline, but it is insufficient for realistic high-dimensional marker data in which marker correlation, linkage disequilibrium, and large p relative to n are central concerns.

2.2 Why the present dataset requires a modern extension

The CSIRO cotton dataset contains thousands of markers but only a few hundred lines. This creates a high-dimensional setting in which multi-marker methods are more appropriate than purely classical single-marker scans. The literature review already identified this issue in its discussion of multiple marker models, collinearity, and shrinkage methods. Accordingly, the present report adopts the following analytical logic:

1. Use single-marker regression as a transparent classical baseline.
2. Use penalised multi-marker regression as the main modern analysis framework.
3. Use simulation anchored on the real genotype matrix to evaluate how different methods behave under realistic marker correlation structures.

3 Data and Setup

3.1 Required packages

```
library(tidyverse)
library(data.table)
library(glmnet)
library(broom)
library(janitor)
library(patchwork)
library(Matrix)
```

3.2 File paths

```
geno_file <- "Genotype_data.txt"
map_file  <- "Genetic_map.txt"
pheno_file <- "Phenotype_data.txt"
```

3.3 Import data

```
geno_raw <- fread(geno_file, header = FALSE)
map_df <- fread(map_file)
pheno_raw <- fread(pheno_file)
```

3.4 Inspect dimensions

```
dim(geno_raw)
```

```
[1] 256 6523
```

```
dim(map_df)
```

```
[1] 6523 3
```

```
dim(pheno_raw)
```

```
[1] 768 4
```

```
head(map_df)
```

	SNP_ID	Chromosome	Position_bp
	<char>	<char>	<int>
1:	Marker-1	A01	373964
2:	Marker-2	A01	432953
3:	Marker-3	A01	506435
4:	Marker-4	A01	514057
5:	Marker-5	A01	514827
6:	Marker-6	A01	528238

```
head(pheno_raw)
```

	Lint_percentage	Seed_index	Lint_index	Seed_oil_content
	<num>	<num>	<num>	<num>
1:	37.5	11.3	8.1	18.2
2:	38.9	9.6	7.2	19.8
3:	41.7	8.5	7.1	19.8
4:	40.9	9.0	7.3	19.8
5:	38.4	10.5	7.9	20.2
6:	36.6	10.2	6.8	19.2

3.5 Add marker names to the genotype matrix

The genotype file has no column names. We use the SNP identifiers from the genetic map.

```
stopifnot(ncol(geno_raw) == nrow(map_df))
```

```
geno_df <- as.data.frame(geno_raw)
colnames(geno_df) <- map_df$SNP_ID
geno_df$Line <- 1:nrow(geno_df)
```

```
geno_df[1:5, 1:8]
```

	Marker-1	Marker-2	Marker-3	Marker-4	Marker-5	Marker-6	Marker-7	Marker-8
1	1	-1	1.000	-1	1	-1	-1	-1
2	1	-1	0.305	-1	0	-1	-1	0

3	0	1	-0.710	0	-1	1	0	1
4	1	0	1.000	-1	0	-1	-1	-1
5	1	-1	0.000	-1	1	-1	-1	-1

3.6

Construct line and season identifiers for phenotype data

The phenotype file contains 768 rows, while the genotype matrix contains 256 lines. This strongly suggests that phenotype observations correspond to three stacked seasons or environments for the same 256 lines.

```
n_lines <- nrow(geno_df)
n_pheno <- nrow(pheno_raw)

n_pheno / n_lines
```

```
[1] 3
```

If the phenotype observations are indeed stacked by season, then each block of 256 rows corresponds to one season.

```
pheno_df <- pheno_raw %>%
  mutate(
    RowID = row_number(),
    Season = case_when(
      RowID <= n_lines ~ "Season_1",
      RowID <= 2 * n_lines ~ "Season_2",
      TRUE ~ "Season_3"
    ),
    Line = rep(1:n_lines, times = n_pheno / n_lines)
  ) %>%
  relocate(Line, Season, RowID)

head(pheno_df)
```

	Line	Season	RowID	Lint_percentage	Seed_index	Lint_index	Seed_oil_content
	<int>	<char>	<int>	<num>	<num>	<num>	<num>
1:	1	Season_1	1	37.5	11.3	8.1	18.2
2:	2	Season_1	2	38.9	9.6	7.2	19.8
3:	3	Season_1	3	41.7	8.5	7.1	19.8
4:	4	Season_1	4	40.9	9.0	7.3	19.8
5:	5	Season_1	5	38.4	10.5	7.9	20.2
6:	6	Season_1	6	36.6	10.2	6.8	19.2

```
tail(pheno_df)
```

	Line	Season	RowID	Lint_percentage	Seed_index	Lint_index	Seed_oil_content
	<int>	<char>	<int>	<num>	<num>	<num>	<num>
1:	251	Season_3	763	43.1	8.4	8.4	21.0
2:	252	Season_3	764	47.8	7.7	9.2	22.4
3:	253	Season_3	765	46.1	7.7	8.6	20.9
4:	254	Season_3	766	44.7	7.8	7.9	23.2
5:	255	Season_3	767	44.7	7.2	7.1	21.4
6:	256	Season_3	768	40.4	8.1	7.1	23.4

3.7 Merge phenotype and genotype

```
analysis_df <- pheno_df %>%  
  left_join(geno_df, by = "Line")  
  
dim(analysis_df)
```

```
[1] 768 6530
```

4 Descriptive Analysis

4.1 Trait names

```
trait_names <- c(  
  "Lint_percentage",  
  "Seed_index",  
  "Lint_index",  
  "Seed_oil_content"  
)
```

4.2 Summary statistics

```
trait_summary <- analysis_df %>%  
  summarise(across(all_of(trait_names), list(  
    mean = ~mean(.x, na.rm = TRUE),  
    sd = ~sd(.x, na.rm = TRUE),  
    min = ~min(.x, na.rm = TRUE),  
    max = ~max(.x, na.rm = TRUE)  
  )))  
  
trait_summary
```

	Lint_percentage_mean	Lint_percentage_sd	Lint_percentage_min	Lint_percentage_max	Seed_index_mean	Seed_index_sd	Seed_index_min	Seed_index_max	Lint_index_mean	Lint_index_sd	Lint_index_min	Lint_index_max	Seed_oil_content_mean	Seed_oil_content_sd	Seed_oil_content_min	Seed_oil_content_max
1	42.44674	2.866789	30.7	49.4	8.48776	0.9811289	6.3	12.3	7.607422	0.5488731	5.9	9.4	19.7099	1.791653	15.2	26.9

4.3 Season-specific summaries

```
season_summary <- analysis_df %>%  
  group_by(Season) %>%  
  summarise(across(all_of(trait_names), list(  
    mean = ~mean(.x, na.rm = TRUE),  
    sd = ~sd(.x, na.rm = TRUE)  
  )), .groups = "drop")  
  
season_summary
```

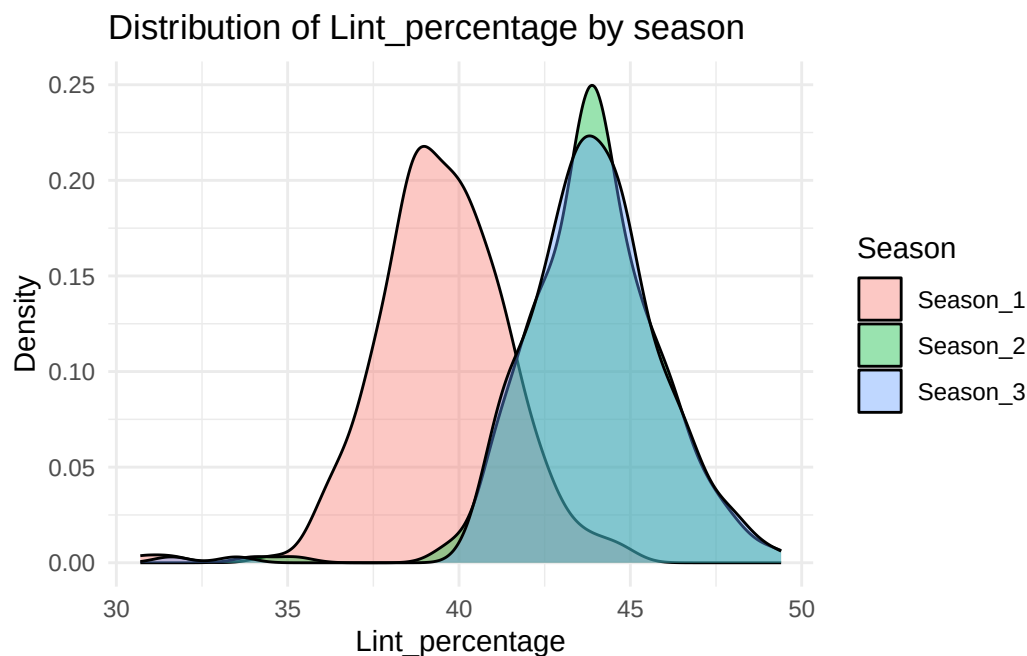
```
# A tibble: 3 x 9
  Season  Lint_percentage_mean Lint_percentage_sd Seed_index_mean Seed_index_sd
<chr>      <dbl>          <dbl>          <dbl>          <dbl>
1 Season_1      39.5            1.92            9.58            0.705
2 Season_2      43.9            1.95            8.15            0.500
3 Season_3      43.9            2.03            7.73            0.499
# i 4 more variables: Lint_index_mean <dbl>, Lint_index_sd <dbl>,
#   Seed_oil_content_mean <dbl>, Seed_oil_content_sd <dbl>
```

4.4 Trait distributions

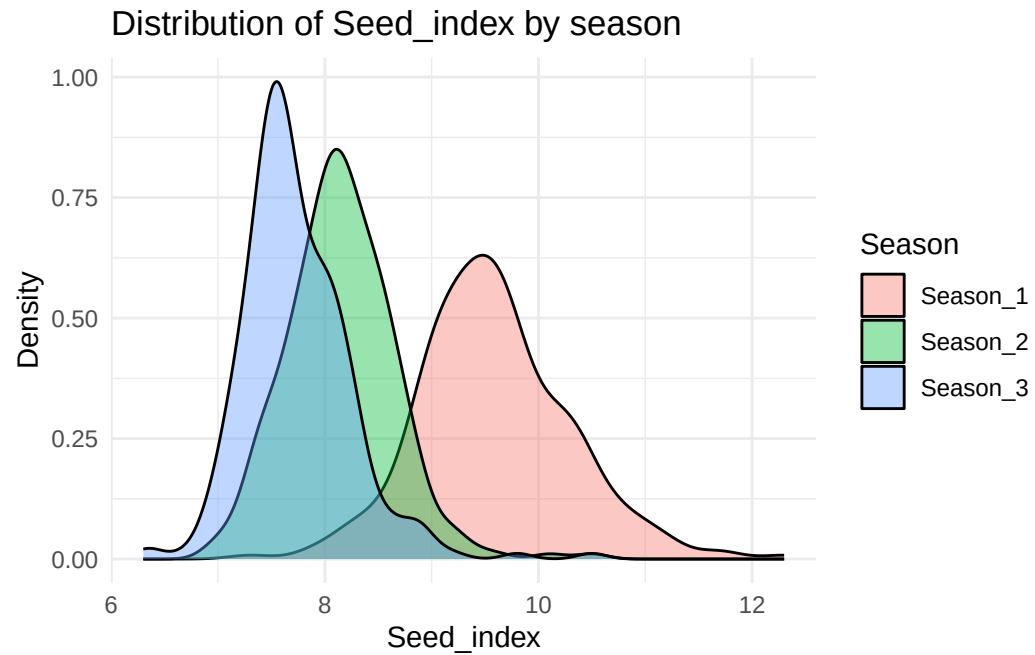
```
plot_trait_density <- function(df, trait) {
  ggplot(df, aes(x = .data[[trait]], fill = Season)) +
    geom_density(alpha = 0.4) +
    labs(
      title = paste("Distribution of", trait, "by season"),
      x = trait,
      y = "Density"
    ) +
    theme_minimal()
}

plots_density <- lapply(trait_names, function(tr) plot_trait_density(analysis_df, tr))

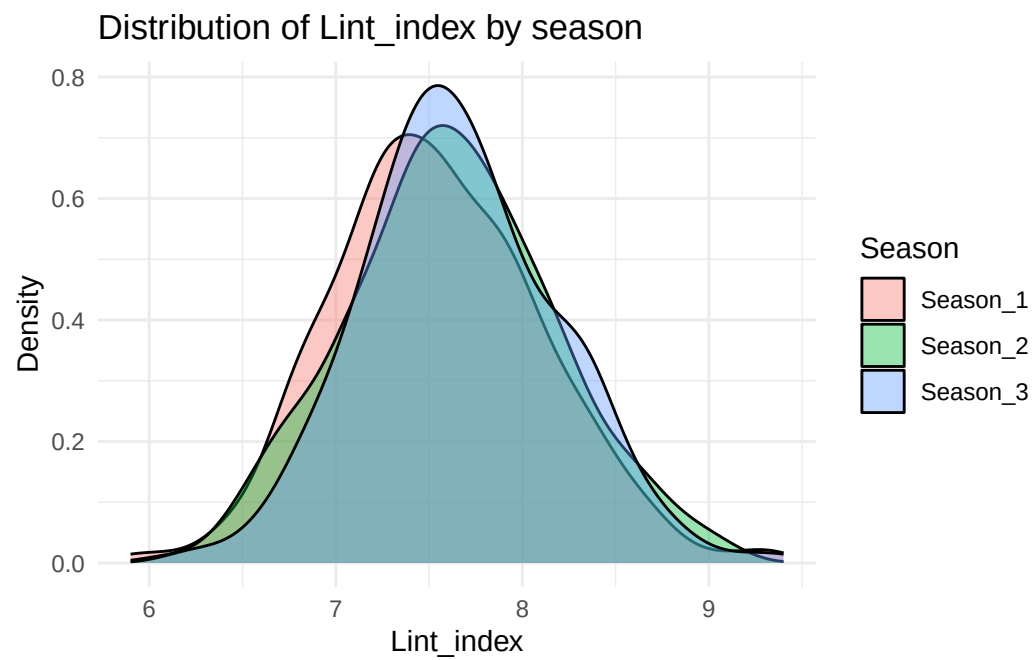
plots_density[[1]]
```



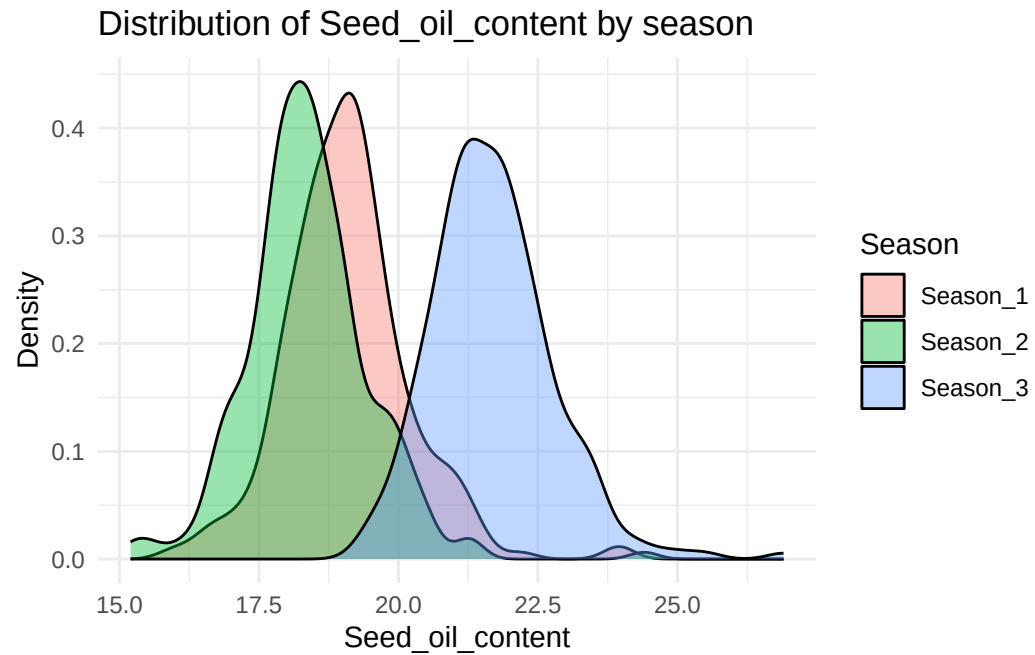
```
plots_density[[2]]
```



```
plots_density[[3]]
```



```
plots_density[[4]]
```

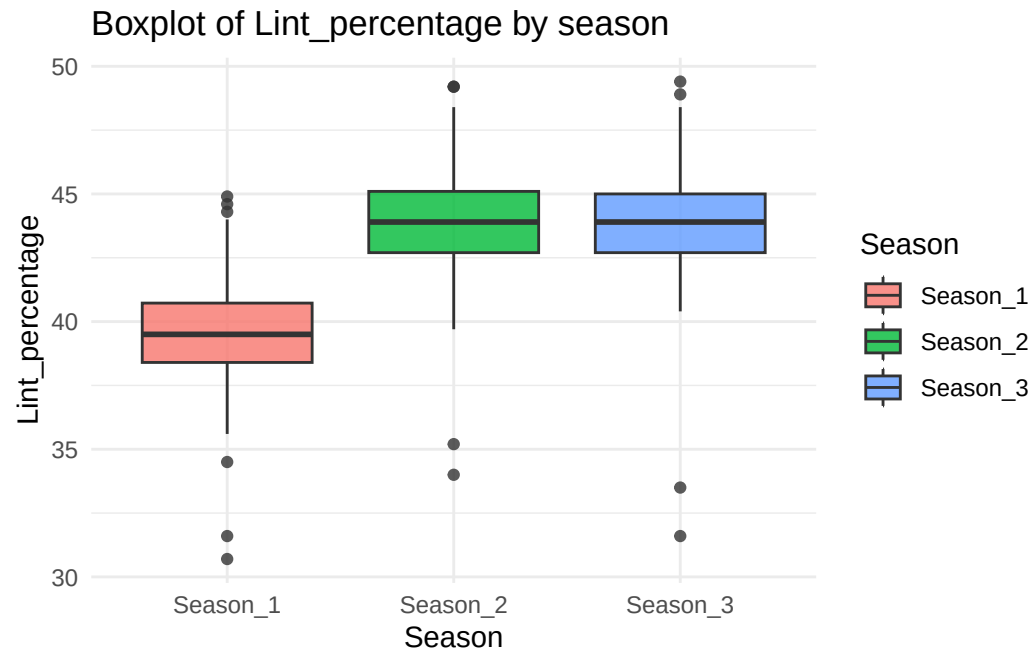


4.5 Trait boxplots by season

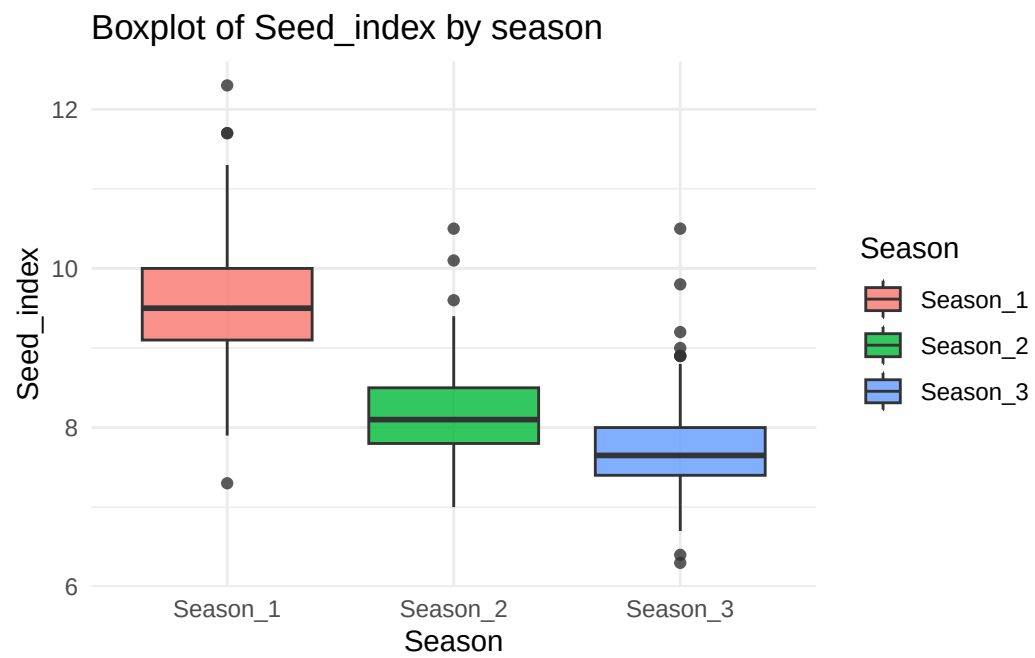
```
plot_trait_box <- function(df, trait) {
  ggplot(df, aes(x = Season, y = .data[[trait]], fill = Season)) +
    geom_boxplot(alpha = 0.8) +
    labs(
      title = paste("Boxplot of", trait, "by season"),
      x = "Season",
      y = trait
    ) +
    theme_minimal()
}

plots_box <- lapply(trait_names, function(tr) plot_trait_box(analysis_df, tr))

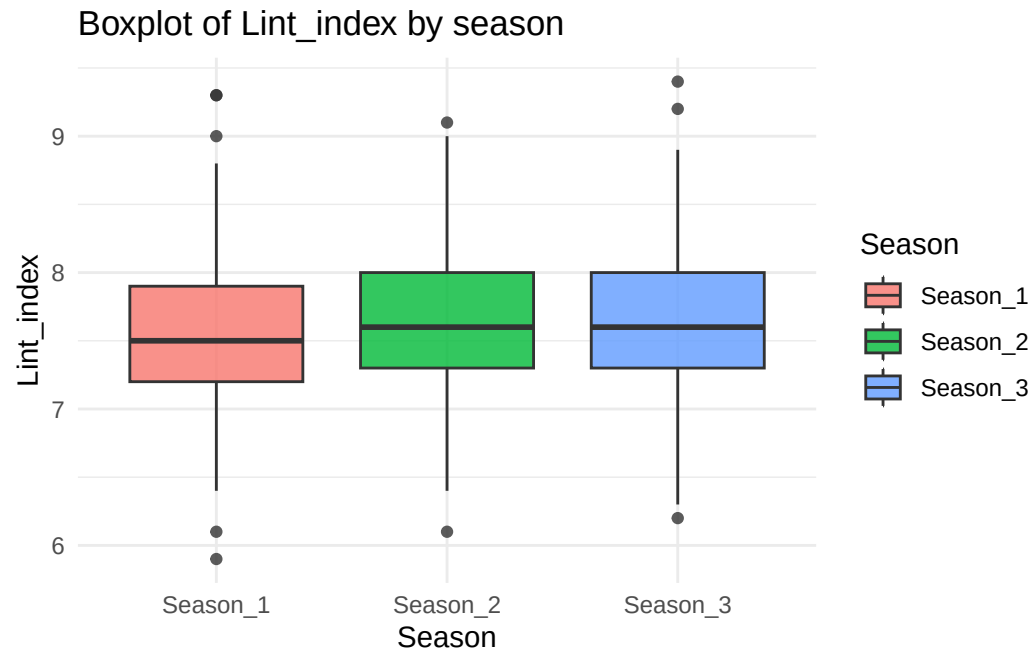
plots_box[[1]]
```

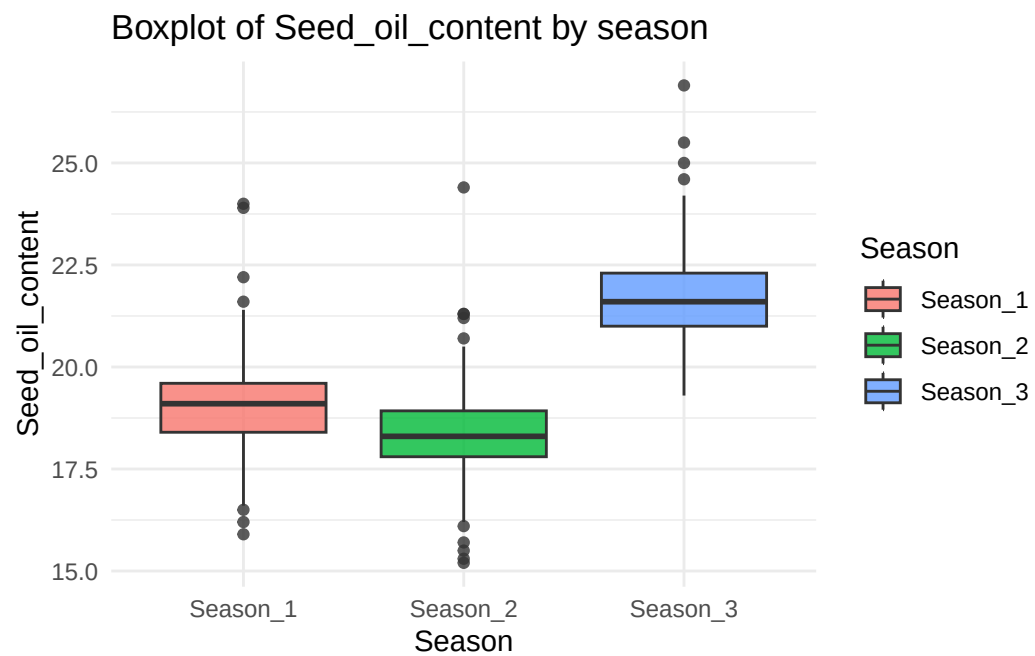
```
plots_box[[2]]
```



```
plots_box[[3]]
```



```
plots_box[[4]]
```



4.6 Correlation among traits

```
trait_cor <- analysis_df %>%
  select(all_of(trait_names)) %>%
  cor(use = "pairwise.complete.obs")

trait_cor
```

	Lint_percentage	Seed_index	Lint_index	Seed_oil_content
Lint_percentage	1.00000000	-0.7783457	0.35414014	0.05702825
Seed_index	-0.77834569	1.00000000	0.14117179	-0.27722812
Lint_index	0.35414014	0.1411718	1.00000000	-0.07199958
Seed_oil_content	0.05702825	-0.2772281	-0.07199958	1.00000000